

Manual Proyecto II – Seamos Civilizados



Sergio Albornoz
Elias Ignacio
Ivan Lahoz
MIX1
Proyecto II
Maig

1. Introducción del proyecto.....	3
2. Análisis y diseño.....	5
2.1. Especificaciones.....	5
2.2. Estructura del proyecto.....	5
2.3. Mecánicas del proyecto.....	5
2.4. Diagrama de Clases y BBDD.....	7
3. Arquitectura i desenvolupament.....	8
3.1. Componentes principales.....	8
3.2. Flujo de datos y eventos del juego.....	9
4. Test y depuración.....	10
Prueba 1: Verificación de conectividad con el Driver de la Base de Dades (SGBD).....	10
Prueba 2: Enrutamiento i persistència de dades en el servidor de producció (Proxmox).....	10
Prueba 3: Integración de módulos iònics amb les interfaces de control (UI/Vistes).....	11
5. Recursos gràficos y de audio.....	12
6. Manual de usuario.....	13
6.1. Instrucciones para ejecutar el juego.....	13
6.2. Controles y objetivo.....	13
7. Reflexión y mejoras.....	15

1. Introducción del proyecto

El proyecto **Civilizations** consiste en el desarrollo de una aplicación interactiva enfocada en la simulación estratégica y la gestión de recursos basados en civilizaciones históricas y fantásticas. En este videojuego, el usuario asume el rol de comandante supremo de una pequeña civilización, teniendo bajo su control la administración y optimización de recursos críticos (Comida, Madera, Hierro y Maná), la investigación de tecnologías y la construcción de infraestructuras clave (Granjas, Carpinterías, Herrerías, Iglesias y Torres Mágicas).

El núcleo dinámico de la aplicación radica en la supervivencia: el entorno es hostil y la civilización se encuentra bajo la constante amenaza de facciones enemigas que buscan su conquista. Esto obliga al jugador a balancear la economía interna con la creación estratégica de un ejército equilibrado, compuesto tanto por unidades ofensivas (Espadachines, Lanceros, Ballesteros y Cañones) como por infraestructuras de defensa activa (Torres de flechas, Catapultas y Torres lanzacohetes) y unidades especiales con propiedades mágicas. El sistema culmina en un complejo motor de resolución de batallas matemáticas y probabilísticas que determina el destino de la región.

Los principales objetivos del proyecto son :

Aplicar el diseño Orientado a Objetos (POO): Implementando una arquitectura de software robusta utilizando conceptos avanzados de POO, tales como la herencia y el uso de interfaces para la gestión estandarizada de las unidades militares y las variables globales del sistema.

Lograr una simulación Probabilística y Balanceo: Desarrollando un motor de combate basado en matrices de probabilidad y estadísticas dinámicas individuales (experiencia, armadura, daño base) que simula enfrentamientos militares realistas y balanceados.

Persistencia de Datos y Auditoría: Integrar una base de datos relacional para almacenar el histórico de las simulaciones, permitiendo el registro detallado de las batallas y la generación de informes técnicos sobre las pérdidas materiales y bajas militares.

Consulta Web: Creando un entorno web dinámico y accesible que actúe como panel de control externo (Dashboard) para que el usuario visualice estadísticas en tiempo real y reportes analíticos de los enfrentamientos.

Gestión de Infraestructura Remota: Desplegar los servicios y componentes del proyecto en un entorno de virtualización profesional, asegurando la alta disponibilidad, la correcta configuración de redes y el aislamiento de entornos de desarrollo y producción.

Las tareas realizadas han sido:

Lenguaje de Programación Principal: Java, empleado para el desarrollo del juego, siendo el motor para el combate y la gestión de los recursos de la civilización a través de estructuras de datos dinámicas ([ArrayList](#)).

Entorno de Desarrollo Integrado (IDE): IntelliJ IDEA / Eclipse, utilizado para la codificación, refactorización y depuración del código fuente en Java.

Sistema de Gestión de Bases de Datos (SGBD):MySQL, mediante el lenguaje **SQL**, encargado de estructurar y securizar las tablas relacionales que guardan el registro de batallas e informes analíticos.

Tecnologías Web: HTML5, CSS3 y JavaScript (junto con frameworks/servlets de backend como Jakarta EE o Spring Boot si aplica), utilizados para el diseño e implementación de la página web dinámica de visualización de estadísticas.

Entorno de Virtualización y Despliegue: Servidor de virtualización **Proxmox** , en el cual se han configurado contenedores independientes para alojar el servidor de aplicaciones web y el servidor de la base de datos relacional.

Control de versiones: Git y GitHub, aplicados para mantener el flujo de trabajo colaborativo del equipo, la gestión de ramas de desarrollo y el control de lanzamientos (*releases*).

2. Análisis y diseño

2.1. Especificaciones

Para poder ejecutar el programa se requiere al menos acceso:

- SO: Windows o Linux
- Acceso al proxmox para poder acceder a la información de la base de datos y poder visualizar los cambios en la página web cada que alguien juega
- Disponer de JDK para poder ejecutar correctamente el programa de java y una IDE como Eclipse
- Librería con conector a BBDD a través de java

2.2. Estructura del proyecto

El proyecto está estructurado en diversos archivos de .java donde se engloban y ejecutan las clases que son necesarias para el correcto funcionamiento del programa.

El archivo Civilizacion.java es donde se recoge todo lo relacionado a la generación de las tropas y estructuras de la civilización, aparte de estar definidas las clases que definen las estadísticas de cada tipo de unidad individual.

Por la otra parte Battle.java es el encargado de gestionar y todo lo relacionado al funcionamiento de las batallas como por ejemplo, los recursos generados al finalizar una batalla, las tropas caídas del enemigo y usuario, más el guardar el desarrollo completo de la batalla en la base de datos.

Mientras que UI.java es quien se encarga de todo lo relacionado con la interfaz gráfica desde donde se ejecuta el proyecto.

Para finalizar tenemos a Main.java que es el archivo encargado de ejecutar las clases y activar los temporizadores de generación de recursos y comienzo de batalla

2.3. Mecánicas del proyecto

Las mecánicas de las que dispone el juego son las siguientes:

- Creación de tropas y estructuras para la civilización:
Las tropas consumen cierta cantidad de recursos a la hora de generarlas y nos permiten tener la posibilidad de participar en las batallas, a su vez cada tipo de tropa tiene una serie de estadísticas base individuales que adquieren al ser creadas. mientras que las estructuras son edificios que aumentan la producción de recursos un X% (Dependiendo del tipo de estructura) cada vez que el contador interno genere los recursos.
- Unidades especiales:
El juego dispone de dos unidades especiales que requieren de maná para ser generadas, los Magos que requiere al menos una torre mágica para ser generados y disponen de gran capacidad ofensiva y alta probabilidad de atacar nuevamente pero defensa nula. Y los sacerdotes que solo puede generarse uno por cada iglesia que se construya y permiten santificar a nuestras tropas aumentando las estadísticas de las mismas mientras haya uno vivo.

	Cost				Stats			
	Food	Wood	Iron	Mana	Armor	Attack Power	Chance of Attack Again	Chance of Generate Wastings
Swordsman	8000	3000	50	0	400	80	3	55
Spearman	5000	6500	50	0	1000	150	7	65
Crosswob	0	45000	7000	0	6000	1000	45	80
Cannon	0	30000	15000	0	8000	700	70	90
Arrow Tower	0	2000	0	0	200	80	5	55
Catapult	0	4000	500	0	1200	250	12	65
Rocket Launcher	0	50000	5000	0	7000	2000	30	75
Magician	12000	2000	0	5000	0	3000	75	0
Priest	15000	0	0	15000	0	0	0	0

Costes y estadísticas base de las unidades

	Cost				Plus generations resources			
	Food	Wood	Iron	Mana	Food	Wood	Iron	Mana
Smithy	5000	10000	12000				+5%	
Carpentry	5000	10000	12000			+5%		
Farm	5000	10000	12000		+5%			
Magic_Tower	10000	20000	24000					3000
Church	10000	20000	24000	10000				

Costes y porcentajes de generación de cada edificio

- Sistema de batalla:

El sistema de batalla que se ha usado, elige quien es el que realiza el inicio de la ofensiva, entonces de forma automática se realizarán los turnos de ataque y defensa hasta que el contador de tropas llegue a una cifra inferior al 20% de tropas que tenía el ejército al momento de iniciar la batalla (Independientemente de si es el jugador o el enemigo), al dar por finalizada la batalla si el usuario resulta vencedor recibirá una cantidad de madera y hierro generada durante la batalla, también el usuario tendrá la oportunidad de poder visualizar un resumen completo del desarrollo de la batalla.

- Tecnologías

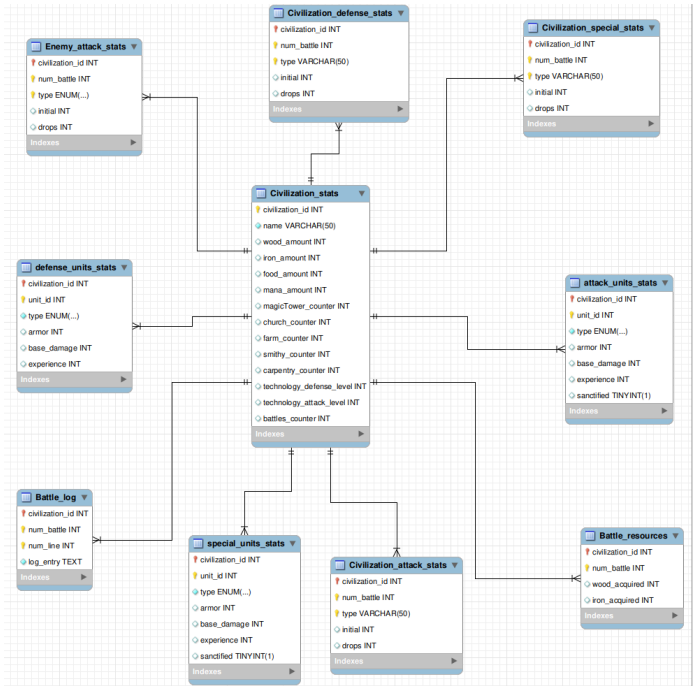
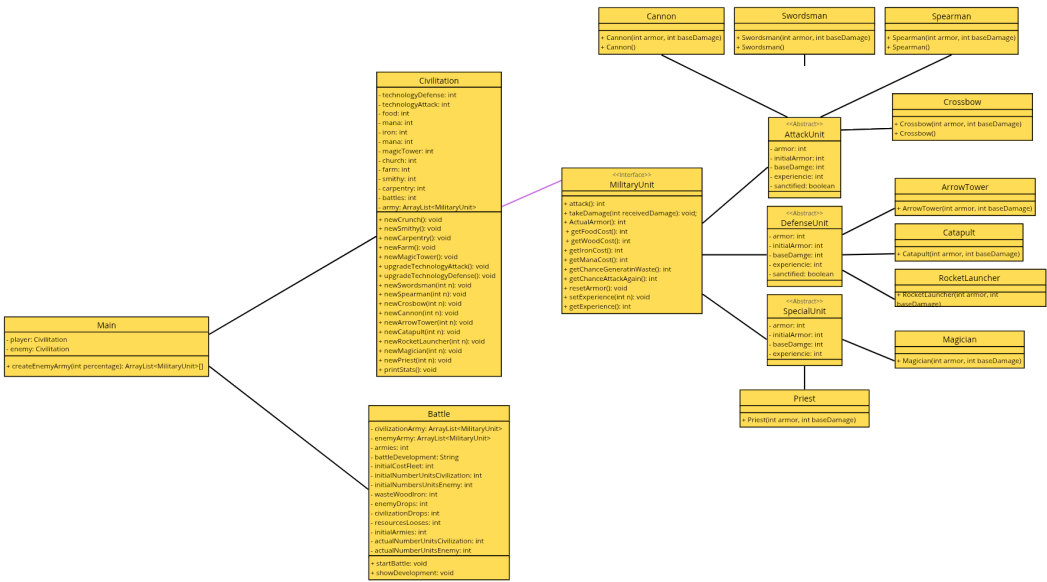
El juego dispone de dos tecnologías principales que permiten aumentar el ataque y defensa de las tropas, ambas tienen un precio inicial en recursos que va aumentando con cada compra.

	Cost		
	Food	Wood	Iron
Defense	100	200	2000
Attack	100	200	2000
Increase cost Defense	+10%	+15%	+20%
Increase costAttack	+10%	+15%	+20%

Costes y porcentajes de aumento de precio de las tecnologías

El cómo el usuario lleva a cabo las acciones de creación y visualización de resumen de batallas se explicará con más detalle en el apartado 6. Manual del usuario.

2.4. Diagrama de Clases y BBDD



3. Arquitectura i desenvolupament

3.1. Componentes principales

Para garantizar la modularidad, escalabilidad y el correcto mantenimiento del software, el sistema *Civilizations* se ha estructurado en un conjunto de componentes interconectados, donde cada clase asume una responsabilidad única dentro del ciclo de vida del videojuego. A continuación, se detallan los módulos principales que componen el núcleo de la aplicación:

- **BaseDatos.java (Capa de Persistencia):** Este componente gestiona el acceso a datos y actúa como nexo de unión entre la aplicación y el sistema de gestión de bases de datos (SGBD). Se encarga de abrir y cerrar las conexiones de forma segura, además de implementar las operaciones necesarias para el registro persistente del historial de las partidas.
- **Battle.java (Motor de Combate):** Clase especializada en la ejecución y resolución de los enfrentamientos dentro del juego. Contiene los algoritmos de simulación de combate, el cálculo de daño basado en los atributos de las unidades y la lógica de turnos. Este módulo procesa las variables de entrada de los ejércitos contendientes para determinar de forma autónoma el resultado de cada batalla.
- **Civilization.java (Modelo de Dominio y Gestión de Recursos):** Representa la abstracción central del modelo de recursos del videojuego. Esta clase es la responsable de instanciar y gestionar tanto la facción del jugador como la facción enemiga. Controla de manera integral el ciclo de vida de las civilizaciones, lo que incluye la administración de sus tropas, la estructura de sus ejércitos y la progresión de sus variables internas.
- **Variables.java (Configuración Global del Sistema):** Este componente actúa como el repositorio centralizado de los parámetros y constantes de configuración del videojuego. Almacena variables críticas para el balanceo del juego (como estadísticas base de las unidades, modificadores de combate o costes de recursos) y ajustes del entorno, permitiendo desacoplar los valores de configuración de la lógica interna del código y facilitando futuras modificaciones o tareas de mantenimiento.
- **UI.java (Capa de Presentación / Interfaz Gráfica):** Componente encargado de la interacción directa con el usuario, implementa la interfaz gráfica de usuario (GUI). Asimismo, se encarga de renderizar visualmente el estado del juego, los menús de navegación y el *feedback* en tiempo real de las acciones del jugador.
- **Main.java (Controlador Central y Lógica de Negocio):** Actúa como el núcleo del bucle de juego. Su función es coordinar e inicializar los demás componentes del sistema, orquestar el flujo de ejecución general de la lógica del juego y asegurar que la comunicación entre las capas de presentación, lógica y persistencia se realice de manera correcta.

3.2. Flujo de datos y eventos del juego

El núcleo de la aplicación se rige por un modelo síncrono basado en eventos periódicos, donde el estado del juego evoluciona a través de dos procesos principales concurrentes: el motor de combate y el sistema de producción de recursos. Ambos procesos dependen directamente de las estructuras de datos globales y del estado actual de la facción.

Las batallas se ejecutan de forma automatizada en intervalos cíclicos de tres minutos. Al iniciarse el evento, el motor realiza una captura del estado del ejército y genera aleatoriamente las del enemigo. Estos datos se contrastan con las matrices de probabilidad y coeficientes analizados en el apartado de configuración para simular el desarrollo del enfrentamiento paso a paso, determinando de manera algorítmica las bajas y el desgaste de las tropas.

Por otro lado, el flujo económico del juego se gestiona mediante un evento de actualización recurrente fijado cada 18 segundos. Esta cadencia temporal es el resultado de un análisis previo de equilibrio de juego, diseñado para optimizar la curva de progresión del jugador sin saturar el rendimiento del sistema. En cada ciclo, el software evalúa los multiplicadores y el nivel de las infraestructuras de la civilización para incrementar proporcionalmente sus fondos de materias primas.

Finalmente, una vez procesados los cambios en el estado del juego, ya sea por la resolución de un combate o por la generación de recursos, se dispara un mecanismo de notificación bidireccional. Por un lado, la capa de presentación o interfaz de usuario recibe los nuevos valores en tiempo real para actualizar los componentes visuales, barras de recursos y paneles de registro de batalla. Por otro lado, el gestor de bases de datos, ejecuta de manera asíncrona las operaciones de actualización necesarias para garantizar la integridad de la información y salvaguardar los registros del juego.

4. Test y depuración

Prueba 1: Verificación de conectividad con el Driver de la Base de Dades (SGBD)

- **Objetivo de la prueba:** Validar que el entorno de ejecución de Java/Node (o el backend correspondiente) pudiera comunicarse con el motor de la base de datos relacional y mapear la información.
- **Problema detectado:** Al desplegar la aplicación en el servidor virtualizado dentro de **Proxmox**, el sistema lanzaba una excepción crítica indicando que el *Driver de conexión SQL* no se encontraba disponible o no era detectado por el entorno de ejecución. Tras una auditoría del control de versiones, se descubrió que las constantes operaciones de sincronización (`git pull` / `git merge`) en GitHub provocan conflictos de rastreo en las dependencias locales, omitiendo o corrompiendo el archivo binario del driver en la carpeta de recursos globales del servidor.
- **Solución aplicada:** Se restauró en el paquete completo nuevamente el driver de mysql para asegurar que las librerías esenciales y conectores queden blindados, persistentes y correctamente indexados en cada despliegue automatizado.

Prueba 2: Enrutamiento i persistència de dades en el servidor de producció (Proxmox)

- **Objetivo de la prueba:** Asegurar el libre flujo de peticiones HTTP/consultas de datos entre la interfaz del panel de control web y las tablas de almacenamiento relacionales del servidor asignado en la infraestructura remota.
- **Problema detectado:** Durante las primeras conexiones en vivo, la aplicación web fallaba de forma persistente. El frontend arrojaba pantallas de error debido a que los servicios internos del servidor no lograban localizar el origen de la base de datos o bien alegaban que los conjuntos de datos de prueba retornaban vacíos (*null pointer/fetch errors*). El problema radicaba en que el backend intentaba resolver la conexión apuntando a direcciones locales (`localhost` / `127.0.0.1`), las cuales no son válidas en un entorno de red segmentado por contenedores.
- **Solución aplicada:** Se procedió a la reconfiguración técnica del archivo de inicio e inicialización del servidor (el punto de entrada `app.js`). Se reemplazaron los direccionamientos locales genéricos por la ruta definitiva y la dirección IP estática asignada al contenedor de la base de datos dentro del nodo de Proxmox. Esto abrió los canales de red correctos, permitiendo que la web renderizara en tiempo real los registros e informes de las batallas.

Prueba 3: Integración de módulos lógicos con las interfaces de control (UI/Vistas)

- **Objetivo de la prueba:** Comprobar la correcta cohesión y transferencia de información entre las clases lógicas estructurales de la simulación de juego (recursos, cálculo de tropas y tecnologías) y los componentes visuales encargados de mostrar los resultados.
- **Problema detectado:** Al ejecutar las pruebas de integración en las vistas principales, el sistema generaba constantes conflictos de inconsistencia de datos y campos vacíos al renderizar zonas muy específicas de las pantallas de simulación. Al rastrear los fallos con el depurador (*debugger*), se constató que los errores no eran fallos de lógica de cómputo, sino que se debían al desarrollo asíncrono del equipo: la interfaz llamaba a ciertos métodos globales cuyas dependencias o clases secundarias aún se encontraban en fase de desarrollo o sin finalizar, rompiendo los contratos de las interfaces de software.
- **Solución aplicada:** Se realizó una sesión intensiva de refactorización y revisión de código. Se dieron valores de prueba en las clases afectadas (solo hasta que la parte que hace el trabajo esté terminada). Esto permitió que las interfaces de prueba operaran con fluidez y estabilidad mientras el resto de los módulos lógicos eran completados de forma progresiva.

5. Recursos gráficos y de audio

Dado que el núcleo del simulador se ejecuta mediante la consola de comandos en Java y el tratamiento de datos se realiza en la base de datos, el proyecto no requiere de una biblioteca de efectos de sonido (archivos .mp3 o .mp4). Sin embargo, se han gestionado y seleccionado cuidadosamente los siguientes recursos visuales y tipográficos:

- **Tipografía de la Interfaz Web:** Para el desarrollo de la plataforma web de informes de batalla y analíticas, se han integrado fuentes tipográficas a través de la biblioteca de *Google Fonts*. Estas fuentes cuentan con la **Licencia Apache 2.0 / Licencia MIT**, lo que permite su uso de forma libre, gratuita y abierta tanto para entornos de desarrollo como de producción.
- **Tipografía del Manual y Documentación:** Para la redacción del manual técnico y elementos corporativos del juego, se ha optado por fuentes estándar del sistema que garantizan la legibilidad y la homogeneidad en cualquier sistema operativo.

Y en lugar de recurrir a bancos de imágenes comerciales o diseñadores gráficos externos, el equipo ha optado por la innovación tecnológica mediante el uso de herramientas de **Generación de Imágenes por Inteligencia Artificial** (como copilot, gemini, chat gpt) para la concepción de toda la identidad visual del proyecto.

- **Logotipo y Portada del Proyecto:** Se diseñó una ilustración conceptual que representa la gestión de territorios y el conflicto bélico entre culturas.
- **Iconografía de Recursos y Edificios:** Se han generado representaciones visuales individuales para los cuatro recursos del juego (Comida, Madera, Hierro y Maná) , así como para los edificios de producción e iglesia (Granjas, Carpinterías, Herrerías, Iglesias y Torres Mágicas).
- **Diseño de Unidades Militares:** Creación de los avatares e ilustraciones conceptuales de los ejércitos para dar identidad a las unidades ofensivas (Espadachines, Lanceros, Ballesteros, Cañones) , defensivas (Torres de flechas, etc) y especiales (Magos y Sacerdotes).
- **Derechos de Uso (Copyright):** De acuerdo con los términos de servicio de las plataformas de IA utilizadas (como OpenAI o Midjourney, gemini, chat gpt), las imágenes generadas bajo sus herramientas de suscripción o términos de uso públicos permiten la explotación y uso de los elementos en proyectos de carácter académico y demostrativo, quedando libres de reclamos de derechos de autor tradicionales.

Y por último la integración de estos componentes difiere según la plataforma final de visualización:

1. **Integración en la Web:** Las imágenes y logos han sido optimizados, escalados y exportados a formatos web eficientes (principalmente .png con transparencia y .webp para reducir los tiempos de carga). Estos archivos se almacenan en el

directorio de recursos estáticos (/public/imagenes/) del servidor web montado en Proxmox. Se mandan a llamar dinámicamente mediante hojas de estilo **CSS**, **HTML** y **HBS**, para ilustrar los reportes y estadísticas de las batallas guardadas en SQL.

2. **Integración en la Documentación y Manuales:** Las ilustraciones de alta resolución han sido maquetadas directamente en los documentos de presentación y guías de usuario para facilitar la comprensión del entorno del videojuego y romper la monotonía del texto técnico.

6. Manual de usuario

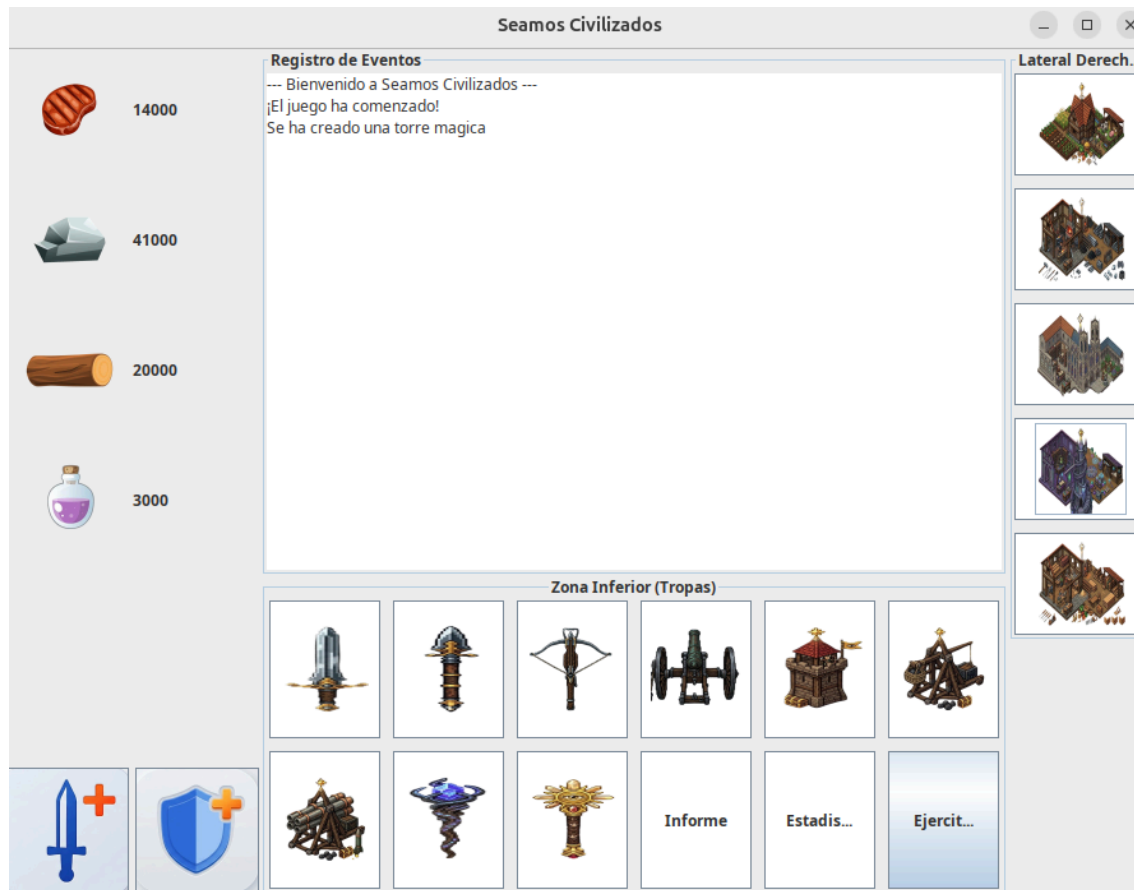
6.1. Instrucciones para ejecutar el juego

Para poder ejecutar el juego se requieren cumplir los requisitos especificados en el apartado 2.1.

Una vez cumplidas las condiciones a través del archivo Main.java presiona el boton de ejecución y el juego debería iniciarse sin problemas

6.2. Controles y objetivo

Una vez iniciado el juego se abrirá una ventana como esta:



Esta ventana es desde donde todo el juego se ejecutara y mostrará los diversos eventos.

El lateral izquierdo de la pantalla mostrará en todo momento la cantidad de cada recurso de los cuales se disponen, en la parte inferior de dicho lateral se encuentran los botones que nos sirven para mejorar las tecnologías.

En la zona inferior se encuentran los botones de generación de unidades, cada vez que se presiona uno se abrirá una ventana que preguntará qué cantidad de unidades de dicho tipo se desean crear, si no se pueden generar todas las unidades se envía un aviso en pantalla junto a la cantidad de unidades que sí se pudieron generar. Los tres botones que no generan unidades se encargan de mostrar en pantalla el informe de la última batalla, estadísticas de la civilización y el ejército enemigo.

La zona lateral derecha son los botones de generación de estructuras, cuando se pulse uno se mostrará un mensaje que indica si se generó o no hay recursos para realizar dicha acción.

En la zona central es el lugar donde se mostrarán los mensajes de todos los eventos que sucedan a lo largo de la partida.

El objetivo principal es gestionar tu propia civilización y alzarse con la victoria antes los múltiples enemigos que trataran de atacarla

7. Reflexión y mejoras

Durante el desarrollo del proyecto **Civilizations**, el equipo técnico se enfrentó a diversos retos de carácter analítico, arquitectónico e infraestructural. A continuación, se detallan las problemáticas principales y las estrategias empleadas para su resolución:

1. Interpretación y ambigüedad de los requerimientos técnicos:

- *Dificultad:* La principal barrera inicial residió en la asimilación del pliego de condiciones (documentación en PDF). En ciertos bloques —particularmente en el modelado estructural de las clases, herencias y dependencias— se detectaron instrucciones que planteaban soluciones iniciales que posteriormente debían ser refactorizadas o descartadas, lo que indujo a confusión durante las fases tempranas de diseño conceptual.
- *Solución:* Para mitigar este problema, el equipo recurrió a la contrastación externa, realizando sesiones de análisis conjunto con compañeros y consultas al cuerpo docente. Esto permitió unificar criterios, clarificar el propósito del patrón de diseño solicitado y consolidar un diagrama de clases definitivo antes de avanzar en la codificación masiva.

2. Acoplamiento de la lógica con las interfaces de control:

- *Dificultad:* Integrar la robusta lógica interna programada en Java (como los motores probabilísticos de combate o el procesamiento de excepciones de recursos) con los entornos de ejecución e interfaces iniciales de prueba supuso una alta complejidad. La sincronización de estados dinámicos del juego en una interfaz unificada fue un reto crítico de arquitectura de software.
- *Solución:* Se optó por un enfoque de desarrollo modular, aislando las pruebas del motor matemático mediante un entorno de depuración controlado en la clase **Main**. Una vez validados los algoritmos de simulación y las colecciones indexadas de unidades (**ArrayList**), se procedió a su enlace con los sistemas externos de reporte

3. Inconsistencia de datos y persistencia en la plataforma Web:

- *Dificultad:* Durante la fase de despliegue del panel de control web en el entorno virtualizado de Proxmox, surgieron errores severos derivados de la persistencia de datos con SQL. En múltiples ocasiones, el sistema fallaba al intentar almacenar registros y estadísticas de batallas de prueba incompletas o erróneamente formateadas. Al no recibir los conjuntos de datos esperados, el *backend* del sitio web lanzaba excepciones críticas, interrumpiendo la ejecución del código del servidor.

- **Solución:** Se implementó revisión profunda de la base de datos. Se depuraron los scripts SQL y se sanearon las transacciones de inserción para garantizar que cualquier fallo en el almacenamiento fuese gestionado de forma controlada sin comprometer la estabilidad global del servidor web.

4. Mejoras futuras y nuevas funcionalidades

Con el objetivo de dotar a **Civilizations** de una mayor profundidad estratégica, escalabilidad y un balanceo de juego óptimo en futuras versiones, se proponen las siguientes líneas de mejora técnica y organizativa:

- **Evolución del balanceo y mecánicas del juego:**
 - **Gestión de costes de mantenimiento:** Introducir un sistema de consumo periódico de recursos. Por ejemplo, exigir que el mantenimiento activo de tropas básicas como *Swordsman* o *Spearman* deduzca automáticamente una tasa de Comida por ciclo, añadiendo mayor complejidad a la macroeconomía de la civilización.
 - **Optimización de Unidades Especiales:** Restringir la capacidad de santificación de la unidad *Priest* (Sacerdote) para que afecte únicamente a un número limitado de tropas en lugar de bendecir a todo el ejército de manera indefinida.
 - **Limitación Tecnológica:** Establecer un tope o nivel máximo (*hard cap*) a las investigaciones de ataque y defensa para evitar curvas de poder exponenciales que rompan el equilibrio estratégico.
 - **Dinámicas del Ejército Enemigo:** Implementar una inteligencia artificial para la civilización atacante de modo que genere recursos de forma paralela y sus oleadas de asalto varíen en tiempos aleatorios y composiciones de ejército basadas en su propia economía.
- **Optimización de la metodología de desarrollo y balance de equipo:**
 - Al realizar una retrospectiva sobre la organización de un equipo integrado por tres desarrolladores, se identificó un cuello de botella metodológico. Al contar con dos perfiles especializados en el desarrollo lógico (Java) y un único miembro dedicado al diseño de la base de datos y la interfaz web, se generaban dependencias e ineficiencias cuando el bloque de programación requería soporte inmediato.
 - Como mejora de gestión de proyectos, se propone acelerar y finalizar en fases tempranas los bloques correspondientes a Lenguajes de Marcas y Bases de Datos. Esto permitirá liberar recursos humanos para que el equipo al completo pueda concentrar sus esfuerzos en las fases críticas de la programación orientada a objetos del backend, logrando una transferencia de conocimiento más homogénea.